

Getting Started with the RAISE Simulator

RAISE 2026 · Getting Started

One command to install · a greenhouse world · a robot you can drive — RAISE 2026 Summer School



A Husky rover + UR5e arm + gripper in a simulated greenhouse (Gazebo). Nothing real can break — experiment!

Install everything — one command

RAISE 2026 · Getting Started

Ubuntu 24.04 · clone, run, wait — that is the whole setup

terminal — one-time setup

```
$ git clone https://github.com/aniskoubaa/raise2026-student
$ cd raise2026-student
$ ./start.sh

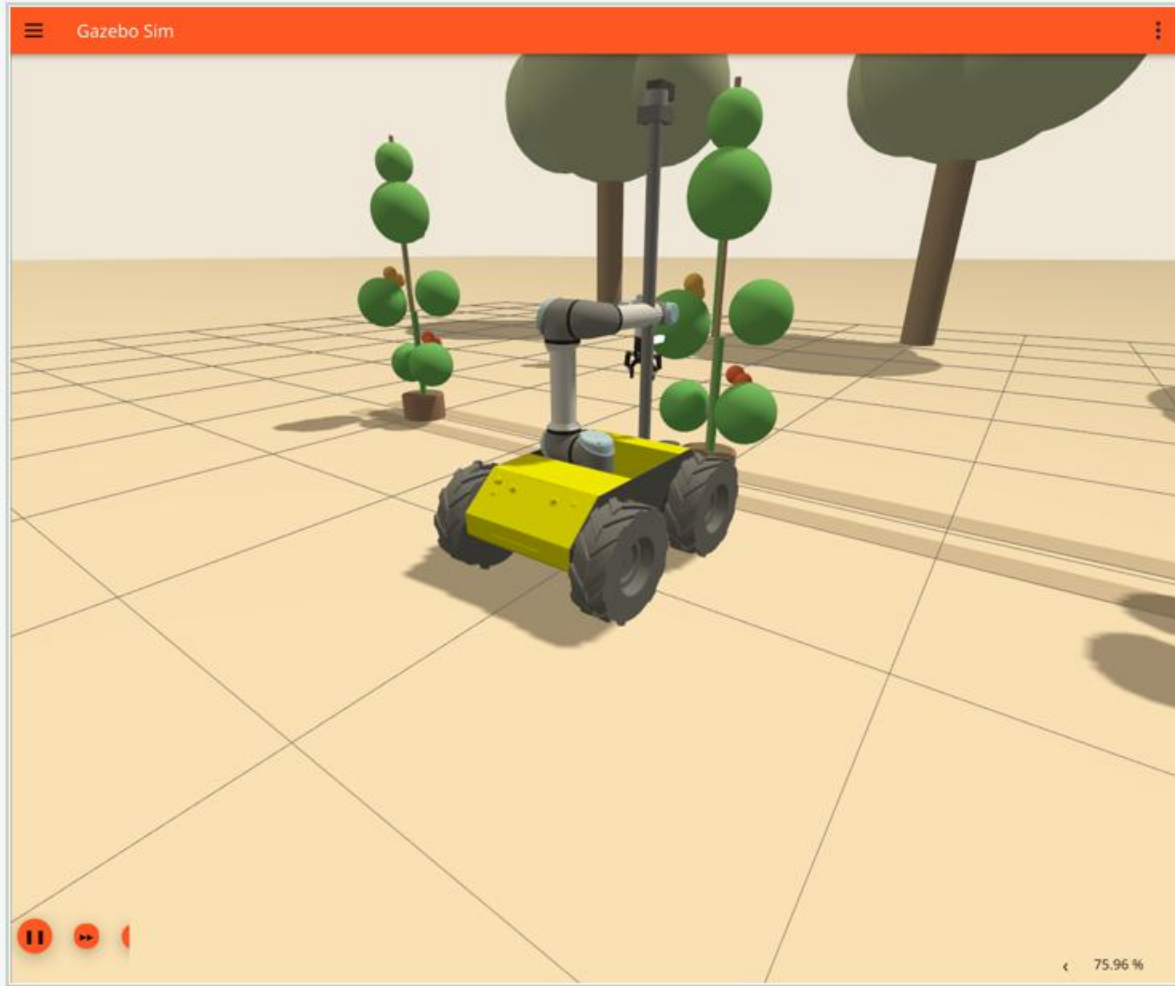
# what start.sh does, automatically:
# 1. installs ROS 2 Jazzy + Gazebo + all packages    (~15 min, once)
# 2. builds the course workspace
# 3. opens the simulator
# 4. starts keyboard teleop HERE + phone teleop at http://<your-ip>:5000
```

Already installed? Skip to the next slide — you just need the world.

Open the world

Three commands you will use every day

RAISE 2026 · Getting Started



```
terminal

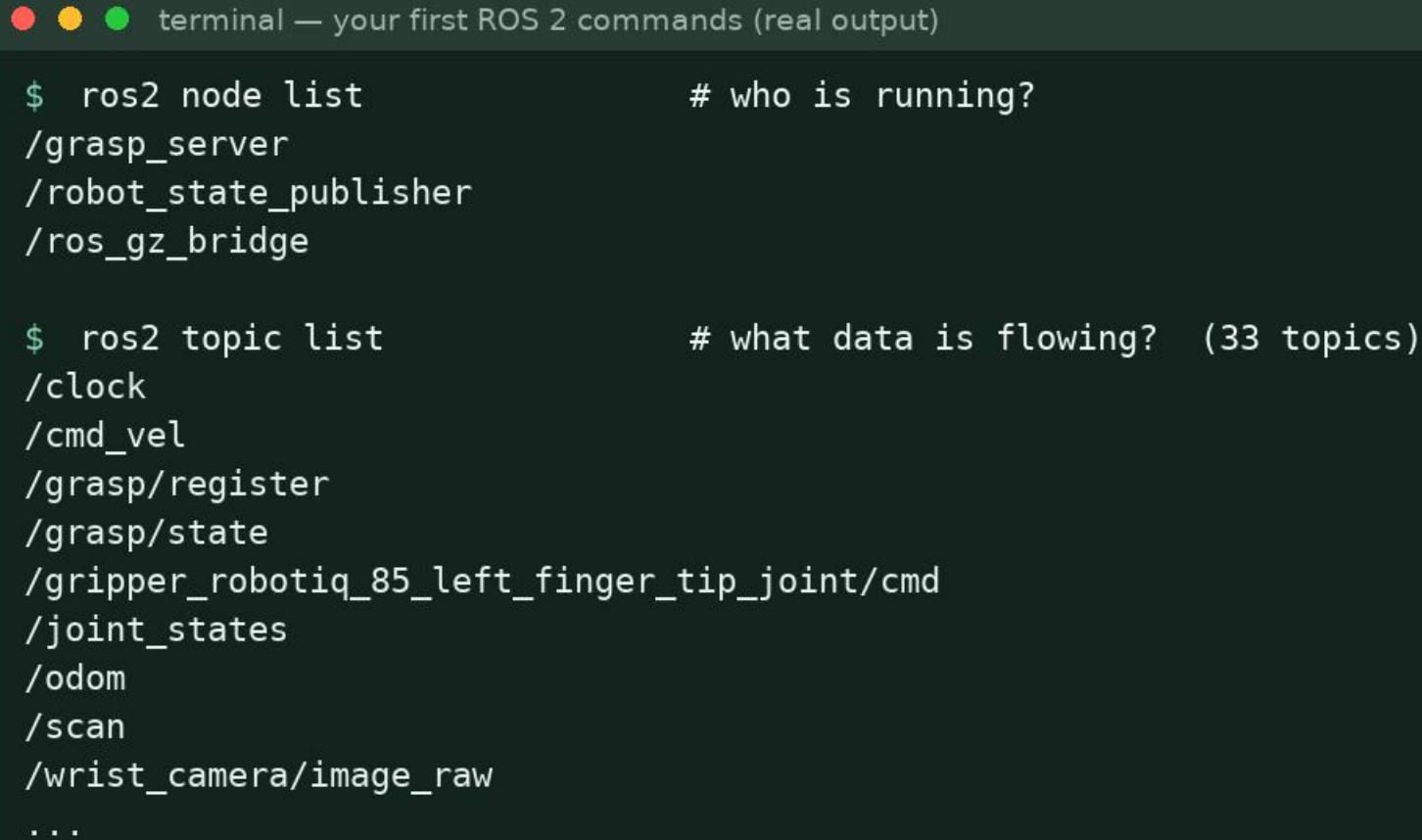
$ raise-sim      # open the world
$ sim_d2         # Day-2 parking spot
$ raise-stop     # stop + reset all
```

raise-sim bundles all the environment fixes (GPU, snap, dialogs) — use it instead of raw gz/ros2 launch.

ROS 2 in two commands

RAISE 2026 · Getting Started

Nodes = the programs · topics = the data streams between them

A terminal window with a dark background and light text. The title bar at the top shows three colored circles (red, yellow, green) followed by the text "terminal — your first ROS 2 commands (real output)". The terminal contains two commands and their outputs. The first command is "\$ ros2 node list" followed by a comment "# who is running?". The output lists four nodes: "/grasp_server", "/robot_state_publisher", and "/ros_gz_bridge". The second command is "\$ ros2 topic list" followed by a comment "# what data is flowing? (33 topics)". The output lists several topics: "/clock", "/cmd_vel", "/grasp/register", "/grasp/state", "/gripper_robotiq_85_left_finger_tip_joint/cmd", "/joint_states", "/odom", "/scan", and "/wrist_camera/image_raw", followed by an ellipsis "...".

```
terminal — your first ROS 2 commands (real output)

$ ros2 node list                # who is running?
/grasp_server
/robot_state_publisher
/ros_gz_bridge

$ ros2 topic list               # what data is flowing? (33 topics)
/clock
/cmd_vel
/grasp/register
/grasp/state
/gripper_robotiq_85_left_finger_tip_joint/cmd
/joint_states
/odom
/scan
/wrist_camera/image_raw
...
```

Everything in the robot — wheels, arm, cameras, gripper — is a topic you can list, read, and publish to.

Read the robot, live

RAISE 2026 · Getting Started

Joint angles and camera rate — straight from the simulator

```
terminal — reading live robot data (real output)

$ ros2 topic echo /joint_states --once      # read ONE message
name:
- front_left_wheel_joint
- ur5e_shoulder_pan_joint
- ur5e_shoulder_lift_joint
- ... (13 joints)
position:
- 1.187e-06
- -1.560
- ...

$ ros2 topic hz /wrist_camera/image_raw    # how fast is the camera?
average rate: 15.092
min: 0.021s max: 0.733s window: 160
```

These two commands are how you debug EVERYTHING: is data flowing, how fast, and what does it say?

Drive it — keyboard or phone

RAISE 2026 · Getting Started

Your first joy: make the rover move

terminal — drive the robot (real output)

```
$ ros2 run raise2026_teleop teleop_keyboard
```

RAISE 2026 — keyboard teleop for the Husky.

w	or	↑	forward
a	or	←	turn left
d	or	→	turn right
x	or	↓	backward
s / SPACE			STOP

Hold the key; the robot moves while you hold it.

```
$ ros2 run raise2026_teleop teleop_phone    # or drive from your PHONE
# open  http://<your-ip>:5000  on the same wifi
```

start.sh launches both automatically. Watch the Gazebo window while you drive.

The alias cheat-sheet

RAISE 2026 · Getting Started

Short nicknames for long commands — all defined in `sim/raise_aliases.sh` (plain bash, open it!)

<code>raise-sim / sim_d2</code>	open the world (default / Day-2 plant-row parking)
<code>raise-stop</code>	stop and reset everything
<code>01_d1 ... 10_d1</code>	Day-1 labs: drive, lidar, camera, PTZ, gripper, navigation...
<code>01_d3</code>	Day-2 teleop (arm + gripper; --task B adds base driving)
<code>05_d3 / 06_d3</code>	auto-record demonstrations / replay an episode
<code>grasp_d3</code>	the grasp service (required for any picking)
<code>get_brain_d4</code>	download + install the trained SmolVLA model
<code>vla_one_d4 / vla_d4</code>	one explained brain call / the full "pick the red tomato" run
<code>finetune_d4</code>	train your own model on your recordings

Alias missing in a terminal? `source <your-repo>/sim/raise_aliases.sh`

```
xdg-open lab_guides/lab1_handson.html
```

Day 1 — sense and act: lidar, camera, gripper, your first agent

```
xdg-open lab_guides/lab2_beginner_handson.html
```

Day 2 (beginner) — command a robot by talking to it, spelled out

```
xdg-open lab_guides/lab2_handson.html
```

Day 2 (full) — dataset, training, evaluation, everything explained

```
xdg-open slides/
```

all lecture + demo slide decks

Trained model: huggingface.co/scalexi/smolvla-raise2026-ripeness-ref (public)